BIG DATA ENGINEERING · REAL-WORLD PIPELINES

Spark Structured Streaming

The concepts you actually need for production streaming pipelines — sources, sinks, watermarking, windows, joins, and fault tolerance.

Kafka → Structured Streaming → Bronze / Silver / Gold

AGENDA

What We'll Cover

The Streaming Model

Unbounded tables, micro-batches & continuous processing

Sources & Sinks

Kafka, files, foreachBatch, and output modes

Time & Triggers

Event time, watermarking, and trigger types

Windows & Joins

Tumbling/sliding windows, stream-static, stream-stream

Fault Tolerance

Checkpointing, WAL, and exactly-once semantics

Real Pipelines

Medallion architecture end to end + monitoring

THE CORE IDEA

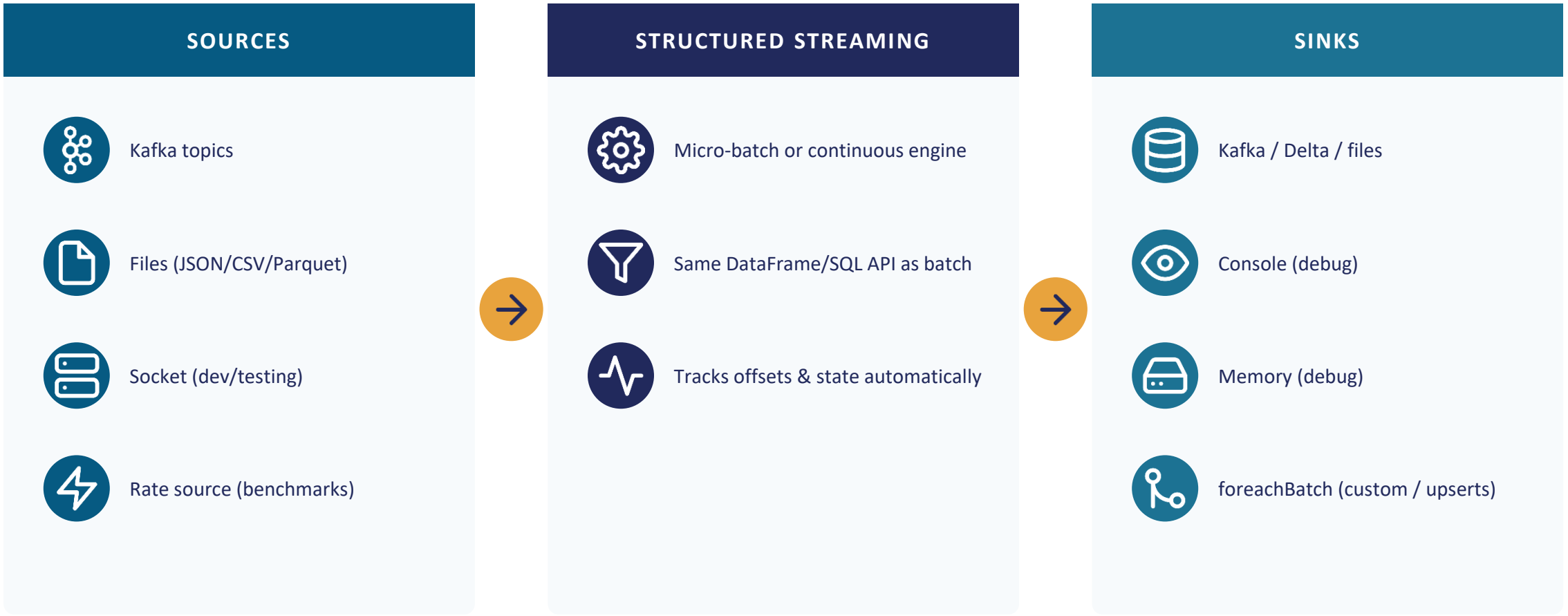
Streams Are Just an Unbounded Table

Structured Streaming treats a live data stream as a table that keeps growing — new rows arrive, but the query you write is the **same DataFrame/SQL API** you already use for batch.

- **Input table:** Every new batch of data appends rows to the conceptual input table.
- **Query:** A DataFrame transformation is applied as if run once over the whole table.
- **Result table:** Spark computes only the incremental result and writes it to the sink.

ARCHITECTURE

Sources, the Engine, and Sinks



Real-world tip: prefer Kafka for production ingestion; use the rate/socket sources only for local testing.

Output Modes: Append, Update, Complete

Append

WHAT WRITES

Only new rows added since the last trigger are written.

WHEN TO USE

Default mode. Works for simple maps/filters and windowed aggregations with a watermark.

Update

WHAT WRITES

Only rows whose aggregate value changed are written.

WHEN TO USE

Streaming aggregations you want to keep refreshing, e.g. running totals per key.

Complete

WHAT WRITES

The entire updated result table is rewritten every trigger.

WHEN TO USE

Small aggregation results, e.g. dashboards over a bounded set of keys.

EXECUTION TIMING

Trigger Types

Default (unspecified)	Runs the next micro-batch as soon as the previous one finishes.	*Lowest latency without a fixed schedule.*
ProcessingTime(interval)	Fires micro-batches on a fixed interval, e.g. every 1 minute.	*Predictable, resource-friendly cadence.*
Trigger.Once / AvailableNow	Processes all currently available data, then stops.	*Cost-efficient scheduled/batch-like streaming jobs.*
Continuous (experimental)	Low-latency, record-at-a-time processing engine.	*~1ms latency needs; limited operator support.*

TIME SEMANTICS

Event Time vs. Processing Time

EVENT TIME

- Timestamp embedded in the record itself (e.g. when a sensor reading or order was created).
- Reflects reality even if data arrives late or out of order.
- Required for correct windowed aggregations & watermarking.

PROCESSING TIME

- The wall-clock time Spark receives and processes the record.
- Simple, always available — but sensitive to pipeline delays.
- Fine for basic monitoring, not for business-accurate aggregates.

Real-world projects nearly always aggregate on event time — a sale, click, or sensor reading "happened" at a fixed moment, regardless of pipeline delay.

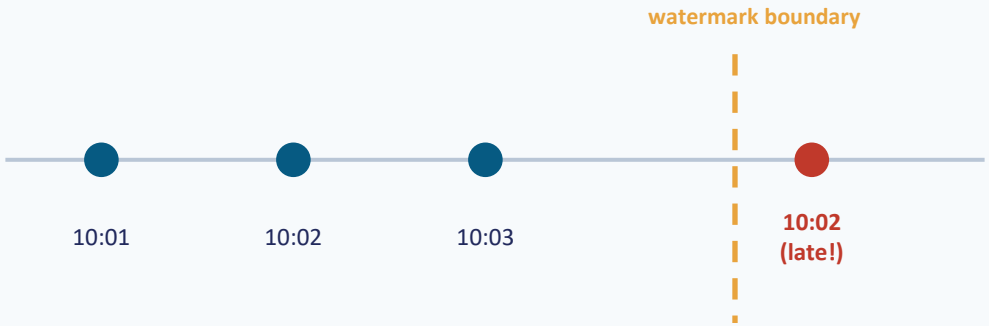
Watermarking

A watermark tells Spark how late data is allowed to be before it stops waiting: ***"drop state older than max(event time) – threshold."***

```
.withWatermark("event_time", "10 minutes")
```

- Bounds how much state Spark must keep in memory for aggregations.
- Lets Append mode emit finalized window results once the watermark passes.
- Records later than the threshold are dropped from that aggregation.

EVENT-TIME TIMELINE



Records left of the boundary are still accepted into open windows; the record arriving at 10:02 after the boundary has passed is dropped.

Interview framing: watermark = "how late can data be" — NOT a scheduling trigger, and it never applies to Complete mode aggregations.

AGGREGATIONS OVER TIME

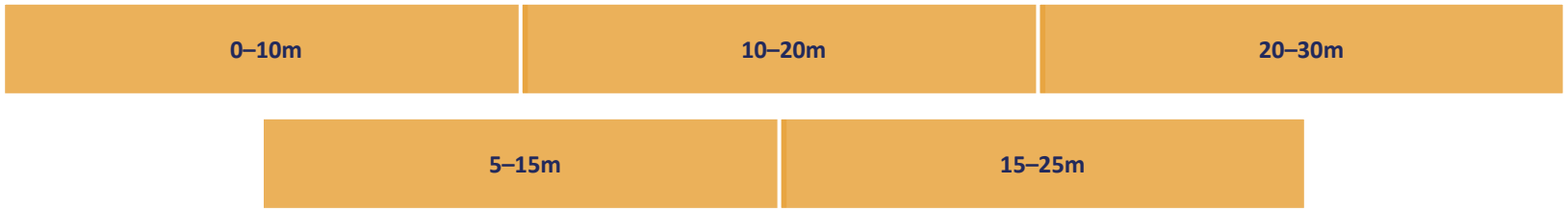
Tumbling vs. Sliding Windows

TUMBLING WINDOWS

window(eventTime, "10 minutes") — fixed, non-overlapping buckets; each event lands in exactly one window.

SLIDING WINDOWS

window(eventTime, "10 minutes", "5 minutes") — overlapping windows; each event can contribute to multiple windows.



Real-world use: tumbling for per-interval KPIs (orders per 10 min); sliding for smoothed/rolling metrics (moving-average revenue).

Stream–Static and Stream–Stream Joins

STREAM <-> STATIC JOIN

Join a streaming DataFrame (e.g. order events) with a static DataFrame **(dim_customers, dim_products)**.

- No watermark needed — static side is fully in memory / re-read each batch.
- Great fit for dimension-table lookups in a star schema.
- Static side changes are only picked up on re-execution unless streamed too.

STREAM <-> STREAM JOIN

Join two live streams, e.g. matching **clicks** with **ad impressions**.

- Requires watermarks on both sides so Spark can bound buffered state.
- Optionally add time-range join conditions to further limit matching.
- Unmatched rows can be emitted via outer joins once past the watermark.

CUSTOM STATE

Arbitrary Stateful Processing

Built-in aggregations cover most cases, but `mapGroupsWithState` / `flatMapGroupsWithState` let you write custom per-key state logic when the built-ins aren’t enough.

Sessionization

Group clickstream events into user sessions with custom timeout logic.

Running Metrics

Maintain arbitrary counters/aggregates per key beyond built-in functions.

Timeout Handling

Explicitly expire state (EventTimeTimeout / ProcessingTimeTimeout) per group.

Interview tip: mapGroupsWithState returns one row per group per trigger; flatMapGroupsWithState can return zero, one, or many rows.

RELIABILITY

Checkpointing & Exactly-Once Semantics

A checkpoint directory is what makes a streaming query resumable and consistent after a failure or restart.

Source offsets

Exactly which records from Kafka/files have been read so far.

Operator state

In-progress aggregation / window / join state (RocksDB or in-memory).

Write-ahead log

Records the intent to process a batch before committing output.

```
.option("checkpointLocation", "/path")
```

EXACTLY-ONCE OUTPUT NEEDS 3 THINGS

1

Replayable source

Kafka offsets (or file source) can be re-read from an exact position.

2

Deterministic processing

The same input batch always produces the same transformed output.

3

Idempotent sink

Re-writing the same batch doesn't duplicate results — e.g. Delta Lake transactions, or upserts keyed by batch id.

THE ESCAPE HATCH

foreachBatch: Batch Power Inside a Stream

`foreachBatch` hands you each micro-batch as a regular batch DataFrame — use the full batch API (writes, upserts, multiple sinks) that streaming sinks don't natively support.

```
def upsert_to_gold(batch_df, batch_id):
    # merge this micro-batch into fact_sales
    delta_table.alias("t").merge(
        batch_df.alias("s"),
        "t.order_id = s.order_id"
    ).whenMatchedUpdateAll()
     .whenNotMatchedInsertAll()
     .execute()

stream_df.writeStream
    .foreachBatch(upsert_to_gold)
    .start()
```

Delta / database upserts (merge on keys)

Writing the same batch to multiple sinks

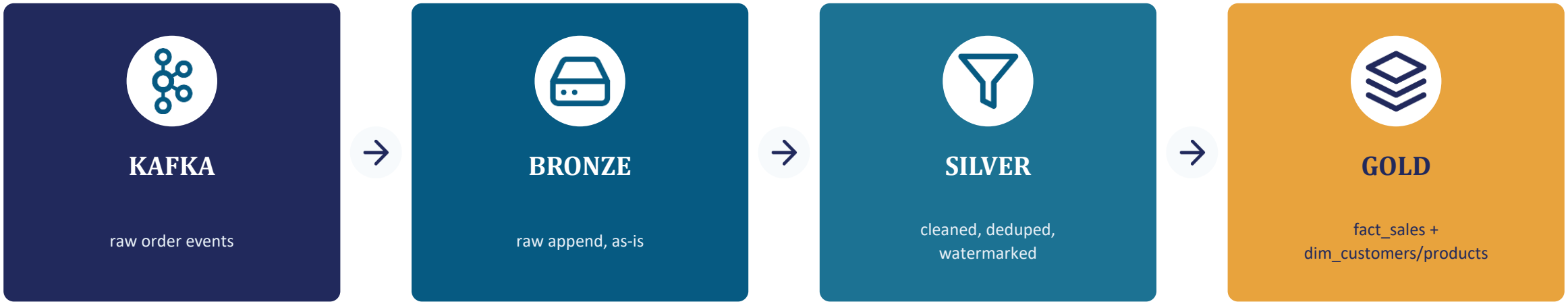
Applying batch-only functions or SCD logic

Custom metrics or side-effects per batch

Note: batch_id is stable across retries — use it to make your writes idempotent.

PUTTING IT TOGETHER

A Real-World Pipeline: Kafka → Medallion



1. Structured Streaming reads Kafka, writes append-only to Bronze Delta tables (source of truth, replayable).
2. A second stream reads Bronze, applies watermark + dedup + type casting, writes to Silver.
3. A third stream (or foreachBatch merge) aggregates Silver into star-schema Gold tables for BI / ML.

OPERATING IN PRODUCTION

Monitoring a Streaming Query

`query.lastProgress` and `query.status` expose metrics you can wire into dashboards and alerts — don’t fly blind in production.

`inputRowsPerSecond`

Incoming data rate — falling sharply may mean an upstream issue.

`processedRowsPerSecond`

Throughput of your query — compare against input rate.

`batchDuration / triggerExecution`

How long each micro-batch takes — watch for growing durations.

`stateOperators.numRowsTotal`

State store size — unbounded growth signals a missing/too-loose watermark.

RECAP

Interview Cheat Sheet

Micro-batch engine

Default execution model; processes data in small batches on a trigger.

Watermark

Bounds lateness & state size: max(eventTime) − threshold.

Output mode

Append (new rows), Update (changed rows), Complete (full table).

Checkpointing

Persists offsets + state for exactly-once, fault-tolerant restarts.

foreachBatch

Drop into the batch API for upserts, multi-sink writes, custom logic.

Stream-stream join

Needs watermarks on both sides to bound buffered state.

You now have the core Structured Streaming toolkit

Sources & sinks • output modes & triggers • event time & watermarking • windows & joins • fault tolerance • foreachBatch • medallion architecture • monitoring

Next step: build a small end-to-end demo — Kafka → Bronze → Silver → Gold — and be ready to talk through every box out loud.